A measurement-based study of physical AI inference

Working notes: setup, facets, and evidence base

P. (Columbia)

September 22, 2026

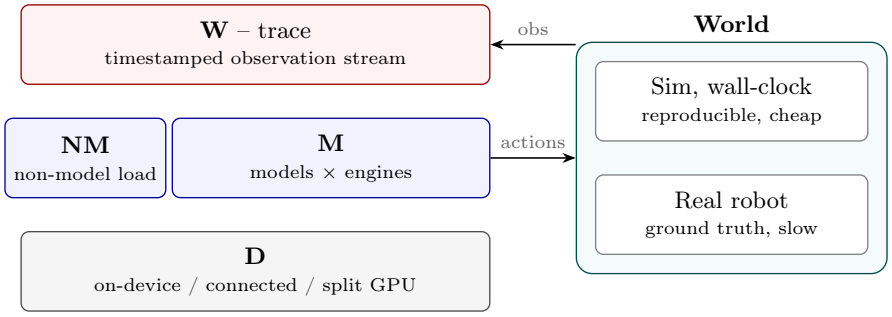
1 Thesis and shape of the study

Physical AI inference (VLAs, world models, and the runtimes around them) is an autoregressive generator feeding a fixed-rate consumer through a stale input channel. The generator emits chunks (action chunks, video blocks); the consumer runs on a clock (actuator loop, display); the input (camera, control events) is always older than the moment the output takes effect. Every published mitigation (RTC, TT-RTC, VLASH, A2C2) is an ML fix for one symptom of that structure, and every published evaluation injects latency as a fixed integer instead of measuring it.

The hypothesis to test: **physical AI inference fails at the interfaces between layers, not inside the model.** Workload↔model is staleness; non-model load↔model is contention; model↔device is transport. If that holds, the fix is a runtime, not another training augmentation.

The study is a measurement grid. Each cell is a configuration of the stack below; each cell yields a failure classification and a task-quality number.

1.1 The stack



W, NM, M, D are all inside the robot. None of them produces observations or receives actions; the World box is the only source of feedback. W is a recording of what the World emitted. The sim sits beside D (it is the environment's counterpart to the device), not above W.

1.2 Three execution modes and what each can measure

Mode	Mechanism	Measures
Trace replay	Stored W fed to M; actions discarded. World off. Open-loop.	Contention, transport, capacity. Latency distribution of M under NM and D. Reproducible.
Wall-clock sim	Sim steps at fixed control rate whether or not M has answered. M runs on the real target device in a separate process over a real transport. Δ is measured per chunk, not injected.	Staleness, deadline misses, task quality. The only mode in which a late action changes the next observation. Reproducible given a recorded Δ trace (replay the Δ sequence into a lockstep sim for statistics).
Real robot	Same loop, physical world.	Ground truth. Validates that the wall-clock sim is not lying. Slow, not reproducible.

Why wall-clock matters: every paper to date runs the simulator in lockstep (pause physics, run inference, inject integer d, resume). Latency is a parameter, not a measurement. The tail of Δ is what causes discontinuities, and lockstep has no tail.

1.3 The grid

Axes: W $\times$ NM $\times$ M $\times$ engine $\times$ D, with the failure vocabulary of §2.5 as the response. Full grid is too large; total grid size to be decided later. A tractable cut: 3 workloads (quasi-static, dynamic, world model) $\times$ NM off/on $\times$ 2 engines $\times$ 3 D = 36 cells. Deliverable: a failure map (which interface breaks first, per cell) plus quality vs. measured-Δ-distribution curves per method, per runtime, per device.

Two axes the ML papers already sweep, kept for comparability: ML method (naive, IT-RTC, TT-RTC, VLASH, A2C2; all implemented in the ETH unified codebase) and runtime policy (blocking loop, dedicated inference thread, two-thread FlashDreams-style runtime, speculative multi-Δ branches).

2 Facets

2.1 W – traces (workloads)

A workload is defined by its timing envelope, not by its model: control rate, reaction budget (how fast the world changes), chunk horizon, quality metric. For measurement it becomes a recorded, timestamped observation stream (replay mode) plus a live sim that generates it (closed-loop mode).

Workload	Rate	Reaction budget	Quality metric	Role / source
Quasi-static manipulation	30 Hz	~300 ms	success rate	Control case; LIBERO regime
Dynamic manipulation	50 Hz	~50 ms	hit rate	Where VLASH's results live; ping-pong, whack-a-mole, conveyor
Navigation among moving agents	10–20 Hz	~200 ms	collision-free completion	Common split deployment; DynaNav
Interactive world model	30–60 Hz	display deadline	dropped frames, input-to-frame latency	FlashDreams' own metrics
Locomotion (WBC)	100 Hz+	hard loop	stability	Not a VLA; always-on load under everything else

2.2 NM – non-model load

Everything sharing the GPU and CPU with M that is never in the papers: VIO/SLAM (cuVSLAM, ORB-SLAM3); camera ISP and decode; image resize and normalize (often on GPU, often the largest hidden cost); depth or point-cloud processing; an object detector or segmenter; a safety monitor; IK/MPC at the control rate; ROS 2/DDS transport; logging; for connected setups, video encode for the uplink. The question for NM is contention: how much co-scheduling inflates M's latency tail relative to M alone. This is an isolation question.

2.3 M – models and engines

Models (span the size and modality axis): $\pi_{0.5}$ (3B VLA; bf16 weights ∼7 GB), Self-Forcing 1.3B (the only FlashDreams model that fits outside 80 GB), Kinetix MLP-Mixer (∼310K params; lower bound, small reactive policy).

Engines (same model, different runtime): LeRobot's loop, openpi's policy server, FlashDreams, TensorRT/Triton, own threaded harness. The engine alone can turn "async" into a stall (see §3.1).

2.4 D – deployment

On-device (Orin), connected (Titan over LAN), split (VLM backbone remote, action expert or small policy local). Use `tc netem` so "connected" is a distribution, not one number.

Hardware notes: Titan RTX (24 GB, Turing) and Titan V (12 GB, Volta) have tensor cores but no native bf16, so fp16 with overflow watch; Titan Xp (Pascal) is a poor fit. VLASH README says LoRA fine-tuning fits under 12 GB. AGX Orin (32/64 GB unified, ∼200 GB/s) is roughly 5× less bandwidth than a 4090, so $\pi_{0.5}$ at ∼150–250 ms per pass is plausible vs. the ∼30 ms the papers design around; at 30 Hz that is $\Delta \approx 5$–8 steps, at the edge of VLASH's `max_delay_steps: 8`. Orin Nano (8 GB) cannot hold $\pi_{0.5}$.

2.5 Failure vocabulary

Fixed before running anything, so the study does not become a pile of latency plots.

- **Staleness**: realized Δ p99 exceeds the workload's reaction budget.
- **Deadline miss**: chunk not ready when the previous one runs out; count stalls and stall duration.
- **Contention**: M's tail under NM load vs. M alone.
- **Transport**: Δ variance attributable to the link.
- **Capacity**: OOM, thermal throttle, sustained-vs-burst latency on Orin.
- **Quality**: task metric at that Δ distribution (the only column the ML papers report).

Reporting: success rate with Wilson CIs; wall-clock time-to-solve (meaningful only under a wall-clock sim); realized Δ p50/p99; stall and deadline-miss counts; chunk-boundary smoothness score.

3 Evidence base

3.1 VLASH (Tang et al., MIT HAN Lab; arXiv 2512.01031v2, Jul 2026)

Mechanism. Asynchronous inference conditions the policy on $(o_t, \hat{s}_{t+\Delta})$ instead of (o_t, s_t), where $\hat{s}_{t+\Delta}$ is the robot state rolled forward under the Δ already-committed actions (delta actions: accumulate; absolute actions: last committed action). Training uses a temporal-offset augmentation: sample $\delta \in \{0, \ldots, \Delta_{\max}\}$, predict $a_{t+\delta:t+\delta+H-1}$ from $(o_t, s_{t+\delta})$; the fixed image forces the model to attend to state. Efficient variant packs one observation and all offset branches into one sequence with a block-sparse mask (for $\pi_{0.5}$: ∼600 obs tokens + ∼50 per offset; 5 offsets = 650→850 tokens, 5× effective batch; 3.26× per-step training speedup, Table IV). Deployment: pick the smallest trained $\delta > \Delta$. Action quantization (q consecutive actions merged) gives 1.5–2.0× completion speedup. Only the proprioceptive half of staleness is fixed; visual staleness is untouched.

Fig. 6 (Kinetix). 2D physics benchmark, 12 dynamic tasks (throwing, catching, balancing). Policy is a 4-layer MLP-Mixer flow policy, $H = 8$, 32 epochs; not a VLA. Δ is injected as integer control steps; future state obtained by stepping the Kinetix sim forward under committed actions with independently sampled noise (estimate, not oracle). Metric: solve rate averaged over 12 tasks, 1,024 rollouts per point, no CIs. Baselines: Sync ($\Delta = 0$, an upper bound), Naive Async, RTC (its compute overhead does not show up as larger Δ here, so its gap is purely algorithmic). Left panel: $\Delta = 1$, $K \in [1, 7]$; Sync itself declines $\sim$0.895$\rightarrow$0.835; Naive is non-monotone (worst at $K = 1$, peaks at $K = 2$). Right panel: $\Delta \in [0, 4]$, $K = \max(\Delta, 1)$ (back-to-back inference, no slack). At $\Delta = 4$: Sync $\sim$0.84, VLASH 81.7%, RTC $\sim$0.60, Naive 51.2%.

Table I (LIBERO). Static tabletop manipulation on a simulated Franka; 4 suites $\times$ 10 tasks. $\pi_{0.5}$ and SmolVLA fine-tuned 30K steps, batch 32, AdamW 5e-5, cosine with 1K warmup; $K = 5$. Sync row is standard fine-tuning (matches Table IV "Original" 96.8); VLASH rows are one offset-augmented model at $\Delta = 1, 2, 3$. Two things differ between rows at once: training augmentation and execution regime. No Naive/RTC rows. Episodes per task not stated; no CIs.

Model	Method (Δ)	Spatial	Object	Goal	Long	Avg
$\pi_{0.5}$	Sync (0)	97.3	99.6	96.7	93.5	96.8
	VLASH (1)	98.8	99.2	96.7	94.4	97.2
	VLASH (2)	97.5	99.2	97.3	94.6	97.1
	VLASH (3)	94.4	98.8	93.3	91.9	94.6
SmolVLA	Sync (0)	81.3	92.9	85.8	55.8	79.0
	VLASH (1)	80.0	92.9	82.3	53.1	77.1
	VLASH (2)	78.5	92.1	86.9	55.0	78.1
	VLASH (3)	79.8	94.2	87.5	54.8	79.1

Kinetix is realistic about dynamics and toy about the model; LIBERO is the reverse; neither measures a latency.

Real-world (the only measured Δ). Fig. 7/8: SO-101 and Galaxea R1 Lite, RTX 5090 laptop, $K = 24$ at 30 Hz, inference delay 2 steps, 20 rollouts per condition. Table II: $\pi_{0.5}$ latency (1 image, $K = 25$, 50 Hz, $D = 500$ ms) H100/5090/4090 = 23.2/29.4/32.3 ms (RTC 48.4/49.6/54.2); max reaction latency $D + 2L$ sync vs. $2L$ async: Sync 546–565 ms, RTC 97–108, VLASH 46–65; speedup 11.8$\times$/9.5$\times$/8.7$\times$. Table III ($q = 2$): ping-pong hit rate Sync 0/20, Naive 0/20, RTC 0/20, VLASH 11/20; whack-a-mole score 3.2/8.6/11.0/28.8.

Code findings (`mit-han-lab/vlash`). The released inference loop is single-threaded and blocking: `launch_next_inference` runs inline at `chunk_index` = `n_action_steps` − `overlap_steps`, and `busy_wait` absorbs the overrun. "Async" means start the forward pass a few steps early conditioned on the chunk's last action (absolute joint positions on SO-101). Works on a 5090 ($\sim$30 ms vs. 33 ms tick); degrades to a stall on anything slower. No inference thread, no CUDA stream separation, no deadline handling. Observation is read only at inference time. Attention is hand-rolled matmul/softmax with explicit mask (materializes $L \times L$; TODO for flashinfer). Prefix KV cached across 10 denoising steps. Only $\pi_0/\pi_{0.5}$ supported; GR00T and SmolVLA results not reproducible from the repo; tightly coupled to LeRobot. `state_cond: true` is the Appendix III projection layer. `max_delay_steps: 8` in the async training config.

3.2 ETH systematic comparison (Agouzoul; arXiv 2605.08168, May 2026; code `TheAyos/async-vla-inference`)

Unified codebases (`bt-kinetix` JAX, `bt-libero` PyTorch/LeRobot v0.4.2) for IT-RTC, TT-RTC, VLASH, A2C2. Kinetix: 10 environments, $H \in \{16, 30\}$, delays to $d = 15$, 2048 rollouts per config, Wilson CIs. LIBERO: SmolVLA, $d \in \{0, 1, 2, 4, 8, 15, 20\}$, $s = 50$, 10 rollouts per task.

Findings relevant here:

- Limitation stated by the authors: d treated as a fixed integer; real deployments vary with GPU scheduling, network jitter, input complexity. Time-to-solve was not reliable in lockstep sim.
- **VLASH on LIBERO uses oracle future states** from the demonstration dataset, following the original authors' implementation (VLASH GitHub issue #10): LIBERO state is 9-D (EE pose + gripper width)

but actions are 7-D (EE velocity + gripper), so rollforward via committed actions is impossible. Table I of VLASH should be read as an upper bound.
- VLASH has a low-delay/high-delay trade-off governed by $[0, d_{\max}]$ (wider $d_{\max}$ costs $\sim$10 pts at low delay on Kinetix); TT-RTC is roughly insensitive to $d_{\max}$ and generalizes out of distribution.
- Kinetix ranking: A2C2 > VLASH ($d_{\max} = 4$) $\approx$ TT-RTC > IT-RTC > naive. LIBERO ranking inverts with delay; A2C2 leads from $d \geq 4$; IT-RTC best at $d = 0$ then collapses toward naive beyond $d = 8$.
- Measured SmolVLA chunk latency on RTX 3090: naive 405.2 ms, TT-RTC 402.7, IT-RTC 469.7 (+16%), A2C2 serial 412.4 (residual head alone 7.27 ms). SmolVLA vision prefill is 89.8% of chunk FLOPs.
- Deployment-latency lookup (from VLA-Perf, π_0, 50 Hz): B100 local 3.2 ms ($d = 0$); RTX 4090 31.1 ms ($d = 1$); Jetson Thor 52.6 ms ($d = 2$); B100+Wi-Fi 7 8.4 ms; B100+5G 27.8 ms ($d = 1$); B100+4G 73.0 ms ($d = 3$); fast cloud 23.4 ms; slow cloud 273.4 ms ($d = 13$).

3.3 Other reference points

- **VLA-Perf** (Jiang et al., arXiv 2602.18397): roofline profiler for VLA inference; where to run (device/edge/cloud) and how hardware and network jointly set end-to-end latency. Produces the latency numbers, never closes the loop to task success.
- **FlashDreams** (NVIDIA): inference/serving runtime for autoregressive video and world models. Two-thread design (model-generation thread and presentation thread meeting through an event buffer and presentation manager), CUDA-graph capture and warmup, ring attention. Reports sustained fps, input-to-frame latency, drops. Requires 80 GB for most models; Self-Forcing 1.3B is the exception. Structurally the runtime VLASH lacks.
- **VSAS-Bench** (arXiv 2604.07634): streaming-VLM benchmark with an asynchronous protocol whose time-aware accuracy folds latency into the score. The right metric shape.
- **AsyncVLA** (arXiv 2602.13476): large VLA on workstation + small model on robot for navigation; enforced latencies to 6 s; concedes network latency over Wi-Fi is hard to evaluate quantitatively.
- **ARLI** (Levine group, arXiv 2608.23831, Aug 2026): RL fine-tuning under inference latency; state augmentation with committed actions and a mid-inference observation.
- **TIC-VLA / DynaNav** (arXiv 2602.02459, ICML 2026): navigation VLA with injected reasoning delays; DynaNav is a dynamic-humans sim suite, one of the few VLA-scale dynamic environments.
- **RTC** (arXiv 2506.07339) and **TT-RTC** (arXiv 2512.05964): origin of the Kinetix 12-task protocol (RPO experts $\rightarrow$ demos $\rightarrow$ flow policy); real-world latency injections of +100/+200 ms.

4 Open questions

- Total grid size and which cells to run first.
- A VLA-scale dynamic sim benchmark barely exists (DynaNav, or a moving-target variant of a manipulation env). The thinness is itself a finding.
- Adaptive δ selection when Δ is a distribution; speculative multi-branch inference using the idle GPU capacity of batch-1 decode.
- Visual staleness: no method addresses it; camera capture$\rightarrow$decode$\rightarrow$H2D is often comparable to the forward pass.
- Deadline-miss policy: what the runtime does when inference overruns the chunk (the released code stalls).